

# Parcial 1 — 2026

Sistemas Operativos / Sistemas Operativos y Redes

Tomás Rodeghiero

UNRC — Universidad Nacional de Río Cuarto

2026

## Resumen

Resolución completa del Parcial 1 del año 2026 de la cátedra de Sistemas Operativos (Lic. en Ciencias de la Computación, UNRC). El parcial consta de cinco ejercicios que abarcan: (i) verdadero/falso sobre arquitectura de SO (monolítico vs. microkernel, traps, modo usuario/-kernel, multitarea, procesos zombie, shell); (ii) linker estático y resolución de símbolos entre dos archivos objeto; (iii) posibles salidas de un programa con `fork()`/`wait()` contrastado con uno multihilo que comparte una variable; (iv) análisis de *deadlock* con grafo de dependencias entre tres tareas y dos recursos; (v) planificación basada en prioridades con *preemption* y *quantum* sobre tres tareas. Cada ejercicio se desarrolla con estructura uniforme: **Enunciado** → **Análisis** → **Desarrollo** → **Respuesta**.

## Índice

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Ejercicio 1 — Verdadero o falso (2 pts)                         | 1  |
| 2. Ejercicio 2 — Linking estático (2 pts)                          | 4  |
| 3. Ejercicio 3 — Salidas posibles: fork vs. threads (2 pts)        | 5  |
| 4. Ejercicio 4 — Análisis de deadlock (2 pts)                      | 7  |
| 5. Ejercicio 5 — Planificación con prioridades y <i>preemption</i> | 9  |
| Resumen del parcial                                                | 10 |

## 1. Ejercicio 1 — Verdadero o falso (2 pts)

**Enunciado.** Marcar verdad (V) o falsedad (F) de las siguientes sentencias. Justificar brevemente las falsas.

### 1.a — SO monolítico

*“Un SO monolítico tiene sus subsistemas o módulos en un único programa. La comunicación entre módulos se hace mediante variables globales e invocaciones a funciones.”*

#### Respuesta

**V.** En un núcleo monolítico (como Linux clásico) todos los subsistemas —scheduler, gestor de memoria, sistema de archivos, drivers, networking— se compilan en un único binario y corren en el mismo espacio de direcciones, en modo kernel. La comunicación entre ellos es directa: invocación de funciones y acceso a estructuras globales (tabla de procesos, tabla de archivos abiertos, listas de páginas libres, etc.).

## 1.b — Microkernel y rendimiento

“Un SO microkernel logra mejor rendimiento sobre uno monolítico debido a su kernel *minimalista*.”

### Respuesta

**F.** Un microkernel típicamente tiene *peor* rendimiento que un monolítico, no mejor. En un microkernel los subsistemas (filesystem, drivers, gestor de memoria) corren como procesos servidores en **modo usuario** y se comunican entre sí por *message passing* (IPC). Cada operación que en un monolítico es una llamada a función dentro del mismo espacio de direcciones, en un microkernel implica al menos dos cambios de modo, dos cambios de contexto y la copia de los datos entre espacios.

Las ventajas del microkernel son **modularidad**, **robustez** (un driver que crashea no tira al kernel) y **seguridad**, pero **no** performance.

## 1.c — La instrucción de trap

“La instrucción de *trap* (usada para hacer un *syscall*) debe ser una instrucción privilegiada.”

### Respuesta

**F.** Justamente lo contrario: la instrucción de *trap* (*syscall* en x86-64, *ecall* en RISC-V, *svc* en ARM) *no* es privilegiada. Si lo fuera, ningún programa de usuario podría usarla, lo cual rompería todo el modelo de *syscalls*.

La instrucción de *trap* es una de las pocas formas legítimas en que un proceso de usuario puede provocar un cambio a modo supervisor. Hardware mediante: la CPU detecta la instrucción, salva el estado, cambia a modo kernel y salta a una dirección fija (la *trap table*) donde el kernel toma el control. Lo *privilegiado* es lo que ejecuta el kernel después del *trap*; la instrucción en sí está diseñada para ser disponible desde modo usuario.

## 1.d — Timesharing y *quantums*

“Un SO timesharing *multiplexa* la CPU entre varias tareas con usos basados en *quantums*.”

### Respuesta

**V.** Un SO de tiempo compartido asigna a cada tarea un cuanto de tiempo de CPU (*quantum*). Al vencer el cuanto, el timer dispara una interrupción que invoca al scheduler para decidir qué tarea ejecutar a continuación. Este es exactamente el modelo *preemptive* sobre el que se construyen Round Robin y todos los planificadores modernos de tiempo compartido.

## 1.e — Multitarea y CPUs

“Un SO multitarea *sólo* es posible con múltiples CPUs.”

### Respuesta

**F.** Multitarea **no** requiere múltiples CPUs. Se puede implementar perfectamente en **una sola CPU** mediante multiplexación temporal (timesharing). De hecho, fue así por décadas antes de la era multicore: Linux, UNIX y Windows corrían multitarea en un único procesador.

La distinción clave es:

- **Multitarea / concurrencia:** *apariencia* de ejecución simultánea de varias tareas. Se logra con una sola CPU multiplexando entre ellas.
- **Paralelismo real:** ejecución *verdaderamente simultánea* de varias tareas. Esto sí requiere múltiples CPUs (o cores).

## 1.f — Proceso *zombie*

“Un proceso *Zombie* ya finalizó su ejecución.”

### Respuesta

**V.** Un proceso zombie es uno que ya terminó (ejecutó `exit()` o fue terminado por una señal) pero cuyo descriptor (PCB) sigue existiendo en la tabla de procesos del kernel porque su *padre* aún no llamó a `wait()/waitpid()` para recoger su exit status. El proceso “está muerto” (no consume CPU, no tiene memoria asignada salvo el descriptor) pero su entrada persiste. Se ve en `ps` con estado Z.

## 1.g — Acceso de un proceso de usuario al reloj

“Un proceso de usuario puede acceder al dispositivo del reloj y programarlo para que genere una interrupción luego de un intervalo de tiempo dado.”

### Respuesta

**F.** Un proceso de usuario **no** puede acceder directamente al hardware del reloj. Los registros del timer y de las IRQs están en el espacio de direcciones del kernel y solo son accesibles en modo supervisor. La MMU bloquea cualquier intento de lectura/escritura directa desde modo usuario.

Lo que el proceso de usuario sí puede hacer es **pedírselo al kernel** vía syscalls: `alarm(N)`, `setitimer`, `timer_create`, etc. El kernel programa el hardware en su nombre y, cuando vence el timer, envía una señal (`SIGALRM`, por ejemplo) al proceso. La interacción es siempre *indirecta*, mediada por el kernel.

## 1.h — El shell

“El shell es un módulo del kernel.”

### Respuesta

**F.** El shell (`bash`, `zsh`, `fish`) es un **programa de usuario** normal y corriente, no parte del kernel. Es simplemente un proceso que corre en modo usuario igual que cualquier otra aplicación: lee comandos por `stdin`, los interpreta, hace `fork()/exec()` para lanzar otros procesos, y maneja redirecciones, pipes y señales *usando syscalls como cualquier otro programa*. No tiene ningún privilegio especial salvo los del usuario que lo lanzó.

## 2. Ejercicio 2 — Linking estático (2 pts)

**Enunciado.** Dados los dos archivos objeto siguientes, enlazados estáticamente con `gcc a.o b.o -o myprog`, determinar si las afirmaciones son verdaderas o falsas.

### Contenido de los archivos objeto

**a.o**

```
.text
0x0000 ...
0x00A0 call 0x00A0
0x00A4 ...
...
// end .text section
.data
0x00E0 ...
.relocation entries
addr type symbol
0x00A0 call f
```

**b.o**

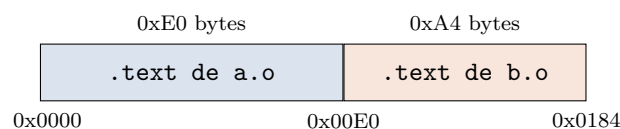
```
.text
0x0000 ...
0x001A f: ...
...
0x00A0 ret
// end .text section
.data
0x00A4 ...
.symbol table
symbol address
f 0x001A
```

### Análisis general

Antes de resolver los tres ítems, recordemos qué hace el linker en un linking estático:

1. **Concatena** las secciones del mismo nombre de todos los archivos objeto. Las `.text` de `a.o` y `b.o` forman, una a continuación de la otra, la `.text` de `myprog`.
2. **Asigna** direcciones absolutas a cada sección concatenada en el ejecutable final.
3. **Resuelve** las referencias externas: para cada entrada en la tabla de relocalaciones, ubica el símbolo en la tabla de símbolos del otro `.o` y completa la instrucción con la dirección final.

**Mapa de direcciones en `myprog`.** Tomando la `.text` de `a.o` primero (tamaño `0x00E0`, ya que `.data` comienza ahí en `a.o`), y luego la `.text` de `b.o` (tamaño `0x00A4`):



Es decir, en `myprog`:

- La `.text` de `a.o` ocupa `[0x0000, 0x00E0)`.
- La `.text` de `b.o` ocupa `[0x00E0, 0x0184)`.

La función `f`, que en `b.o` estaba en el offset `0x001A` dentro de su propia `.text`, queda relocada a  $0x00E0 + 0x001A = 0x00FA$  en `myprog`.

### 2.a — Dirección de `f` en `myprog`

“La dirección de la función `f` en `myprog` resultará `0x001A`.”

#### Respuesta

**F.** `0x001A` es solo el offset de `f` dentro de la `.text` de `b.o`; es una dirección *relativa* en el archivo objeto antes del linking. Al hacer `gcc a.o b.o -o myprog`, el linker coloca la `.text` de `b.o` a continuación de la de `a.o` (que mide `0xE0`). La dirección absoluta de `f` en el ejecutable final es:

$\text{dir}(f) \text{ en myprog} = \text{inicio de b.o.text} + \text{offset interno} = 0x00E0 + 0x001A = 0x00FA.$

## 2.b — `call 0x00A0` es una llamada local?

“La instrucción `call 0x00A0` es una llamada a una función local en `a.o`.”

### Respuesta

**F.** No es una llamada local. La pista definitiva está en la **tabla de relocalaciones** de `a.o`:

```
.relocation entries
  addr type symbol
0x00A0 call f
```

Esta entrada dice: “en la dirección `0x00A0` hay una instrucción de tipo `call` cuyo operando debe completarse con la dirección final del símbolo externo `f`”. El `0x00A0` que aparece como operando del `call` *es un placeholder* (0 o autorreferencia) que se rellena durante el linking. El símbolo `f` **no** está definido en `a.o` sino en `b.o`, por lo cual la llamada es *externa*, no local.

Si fuera local, la referencia ya estaría resuelta en `a.o` y no aparecería en la tabla de relocalaciones.

## 2.c — Tamaño de `.text` en `myprog`

“La sección `.text` de `myprog` será de tamaño `0x00E0 + 0x00A4`.”

### Respuesta

**V** (con la salvedad de alineación que se aclara abajo).

En `a.o`, la sección `.text` arranca en `0x0000` y la sección `.data` comienza inmediatamente en `0x00E0`. Es decir, el tamaño de `.text` en `a.o` es `0xE0` bytes. Análogamente, en `b.o` la `.text` mide `0xA4` bytes (desde `0x0000` hasta donde comienza `.data` en `0x00A4`).

Como el linker concatena las dos `.text` en una sola, el tamaño resultante es la **suma**:

$0x00E0 + 0x00A4 = 0x0184$  bytes.

**Caveat real:** en sistemas reales el linker puede insertar algunos bytes de *padding* para satisfacer requisitos de alineación entre las dos secciones `.text` concatenadas (en x86 típicamente 16 bytes), por lo que el tamaño *exacto* podría ser un poco mayor. Pero la suma es esencialmente correcta y la afirmación es válida en el modelo simplificado del ejercicio.

## 3. Ejercicio 3 — Salidas posibles: `fork` vs. `threads` (2 pts)

**Enunciado.** Mostrar las posibles salidas de los dos programas.

### Programa A (con `fork`)

```
int x = 0;
if (fork() == 0) {
    /* hijo */
    x += 1;
    printf("child:%d ", x);
    exit(x);
} else {
    /* padre */
    printf("parent:%d ", x);
    wait(NULL);
    x++;
    printf("parent:%d ", x);
}
```

## Análisis

`fork()` crea un **nuevo proceso** con su propia copia de la memoria. Después del `fork`, padre e hijo tienen cada uno su propia variable `x` (originalmente 0 en ambos). Lo que uno modifique *no afecta* al otro.

- **Hijo:** hace `x += 1` (su `x` vale 1), imprime `"child:1 "`, hace `exit(1)`.
- **Padre:** imprime `"parent:0 "` (su `x` sigue en 0), entra a `wait(NULL)` hasta que el hijo termine, luego hace `x++` (su `x` pasa a 1) e imprime `"parent:1 "`.

¿Qué puede variar? El orden entre el primer `printf` del padre (`"parent:0 "`) y el del hijo (`"child:1 "`) depende del scheduling. El segundo `printf` del padre (`"parent:1 "`) está después de `wait`, así que siempre va al final.

## Salidas posibles

| Orden                 | Salida                                 |
|-----------------------|----------------------------------------|
| Padre imprime primero | <code>parent:0 child:1 parent:1</code> |
| Hijo imprime primero  | <code>child:1 parent:0 parent:1</code> |

### Idea intuitiva

Lo que **NO** puede aparecer: `"parent:1 "` antes de `"child:1 "`, porque el segundo `printf` del padre está después de `wait(NULL)`, que bloquea hasta que el hijo finalice. Por construcción, `"parent:1 "` es siempre lo último.

## Programa B (con threads)

```
int x = 0;

void f(void) {
    x++;
    printf("t1:%d ", x);
    exit_thread();
}

int main(void) {
    thread t1 = create_thread(f);
    printf("main:%d ", x);
    wait_thread(t1);
    printf("main:%d ", ++x);
}
```

## Análisis

A diferencia de `fork()`, `create_thread()` crea un **hilo dentro del mismo proceso**, que **comparte el espacio de direcciones** con `main`. La variable `x` es una sola: si el hilo `t1` hace `x++`, `main` la ve modificada.

Esto introduce una **race condition** entre la modificación de `x` en `t1` y la lectura de `x` en el primer `printf` de `main`.

Análisis paso a paso:

- `t1` ejecuta: `x++` (queda en 1) → `printf("t1:1 ")` → `exit_thread`.

- `main` ejecuta: `printf("main:?")` (lee el  $x$  actual, que puede ser 0 o 1 según si `t1` alcanzó a incrementar antes) → `wait_thread` (espera fin de `t1`) → `++x` (sube  $x$  a 2) → `printf("main:2")`.

El `++x` del último `printf` se evalúa siempre *después* del `wait_thread`, momento en el cual el incremento de `t1` ya quedó hecho. Por eso el **último valor impreso es siempre 2**.

### Salidas posibles

| Interleaving                                                  | Valor de $x$ que lee main | Salida                          |
|---------------------------------------------------------------|---------------------------|---------------------------------|
| <code>t1</code> corre completo antes que main lea             | 1                         | <code>t1:1 main:1 main:2</code> |
| main lee antes que <code>t1</code> incremente                 | 0                         | <code>main:0 t1:1 main:2</code> |
| <code>t1</code> incrementa, main lee, <code>t1</code> imprime | 1                         | <code>main:1 t1:1 main:2</code> |

### Importante

La diferencia esencial entre los dos programas es la naturaleza de  $x$ :

- En **programa A**, `fork` duplica el espacio de direcciones. Padre e hijo tienen variables  $x$  *independientes*. No hay race condition.
- En **programa B**, los hilos *comparten* el espacio de direcciones. La misma variable  $x$  es accedida concurrentemente. Hay race condition real: el valor que lee `main` depende del orden de ejecución.

Esta es exactamente la diferencia conceptual entre procesos y threads.

## 4. Ejercicio 4 — Análisis de deadlock (2 pts)

**Enunciado.** Dada la siguiente secuencia de operaciones en un sistema concurrente:

```
1: T1: requiere y adquiere R1
2: T2: requiere R1
3: T1: libera R1
4: T3: requiere y adquiere R2
5: T2: adquiere R1
6: T1: requiere R2
7: T2: requiere R2
8: T3: requiere R1
9: ...
```

(a) Determinar si en el paso 8 se produce *deadlock*. Mostrar el grafo de dependencias.

(b) Si el administrador finaliza T2, ¿pueden las demás tareas progresar?

### Análisis: estado al finalizar el paso 8

Reconstruyamos quién posee qué y quién espera qué después de cada paso:

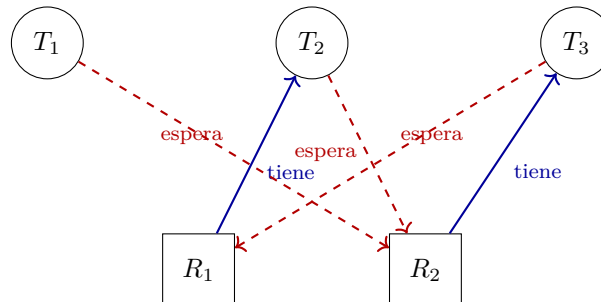
| Paso | Operación                    | R1    | R2 | Bloqueadas     |
|------|------------------------------|-------|----|----------------|
| 1    | T1 adquiere R1               | T1    | —  | —              |
| 2    | T2 requiere R1 (T1 lo tiene) | T1    | —  | T2             |
| 3    | T1 libera R1                 | libre | —  | T2 (despierta) |
| 4    | T3 adquiere R2               | libre | T3 | —              |
| 5    | T2 adquiere R1               | T2    | T3 | —              |
| 6    | T1 requiere R2 (T3 lo tiene) | T2    | T3 | T1             |
| 7    | T2 requiere R2 (T3 lo tiene) | T2    | T3 | T1, T2         |
| 8    | T3 requiere R1 (T2 lo tiene) | T2    | T3 | T1, T2, T3     |

### Configuración tras el paso 8:

- R1 está asignado a T2.
- R2 está asignado a T3.
- T1 espera R2 (lo tiene T3).
- T2 espera R2 (lo tiene T3).
- T3 espera R1 (lo tiene T2).

### Grafo de asignación de recursos

Convención clásica: arista  $R \rightarrow P$  si el recurso está asignado al proceso; arista  $P \rightarrow R$  si el proceso espera el recurso.



#### 4.a — ¿Hay deadlock?

Sí. Sigamos las aristas a partir de T2:

$T2 \rightarrow R2$  (espera)  $\rightarrow T3$  (tiene)  $\rightarrow R1$  (espera)  $\rightarrow T2$  (tiene)  $\rightarrow \dots$

Hay un **ciclo cerrado**  $T2 \rightarrow R2 \rightarrow T3 \rightarrow R1 \rightarrow T2$ . Como los recursos son de instancia única, un ciclo basta para garantizar deadlock. Las dos tareas atrapadas en el ciclo son **T2 y T3**.

**¿Y T1?** T1 espera R2. R2 está retenido por T3, que a su vez está en deadlock. T1 **no forma parte del ciclo** pero está *bloqueada como víctima colateral*: nunca obtendrá R2 porque T3 nunca lo va a liberar. T1 es lo que se llama una *víctima indirecta* del deadlock.

#### Respuesta

(a) Hay deadlock entre **T2 y T3** (ciclo  $T2 \rightarrow R2 \rightarrow T3 \rightarrow R1 \rightarrow T2$ ). **T1** también está bloqueada pero como consecuencia secundaria del deadlock, no como integrante del ciclo.

#### 4.b — Si el admin finaliza T2

Si el administrador finaliza T2 (por ejemplo, con `kill -9`):

1. T2 muere y libera R1 (todo recurso retenido por un proceso se libera cuando éste termina).
2. T3 estaba esperando R1  $\rightarrow$  ahora puede adquirirlo. T3 ahora tiene R1 y R2.
3. T3 progresa, termina su sección crítica y libera R1 y R2.
4. T1 estaba esperando R2  $\rightarrow$  al liberarse, T1 lo adquiere y continúa hasta terminar.

#### Respuesta

(b) Sí, las demás tareas pueden progresar. Finalizar T2 rompe el ciclo del deadlock: T3 obtiene R1 y termina su trabajo, lo que a su vez permite que T1 obtenga R2. Ésta es la técnica clásica de **recuperación de deadlock** matando un proceso del ciclo.



## 5. Ejercicio 5 — Planificación con prioridades y *preemption*

**Enunciado.** Dadas las siguientes tareas, todas creadas en  $t = 0$ :

| Tarea | Prioridad | Comportamiento          |
|-------|-----------|-------------------------|
| idle  | 0         | CPU:1 CPU:1 ...         |
| P1    | 1         | CPU:2.5 EXIT            |
| P2    | 2         | CPU:1 WAIT:1 CPU:1 EXIT |

Scheduler basado en prioridades con *preemption* y  $\text{quantum}=2$ .

**Convención de prioridad.** Bajo la asignación dada, `idle` tiene prioridad 0 (la mínima), P1 prioridad 1 y P2 prioridad 2 (la máxima). *Mayor número  $\rightarrow$  mayor prioridad.*

### 5.a — Orden de la cola RUNNABLE tras el arribo

Al instante  $t = 0$  las tres tareas entran simultáneamente al sistema. La cola RUNNABLE (o READY) se ordena por prioridad **descendente**:

#### Respuesta

(a) Orden de la cola READY en  $t = 0$ :

[P2, P1, idle]

P2 con prioridad 2 toma la CPU primero; P1 (prio 1) queda en segundo lugar; idle (prio 0) al final.

### 5.b — ¿P1 termina antes que P2?

Simulamos paso a paso. Usaremos la notación  $[t_i, t_f)$  = intervalo en que cada tarea corre.

**Paso 1:  $[0, 1)$  — P2 ejecuta su primer CPU:1.** P2 (prio 2) es la de mayor prioridad. Corre 1 unidad. Al llegar a  $t = 1$ , P2 termina su primera ráfaga y entra a WAIT:1 (se duerme por 1 unidad).

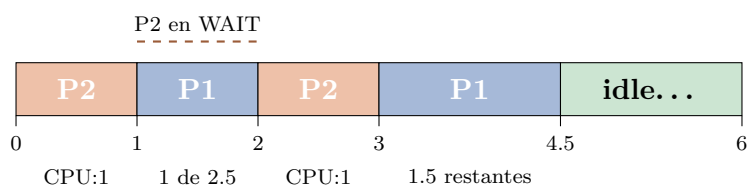
**Paso 2:  $[1, 2)$  — P1 toma la CPU.** Con P2 dormida, la READY queda [P1, idle]. P1 (prio 1) supera a idle (prio 0). P1 ejecuta 1 unidad de su CPU:2.5, le quedan  $2.5 - 1 = 1.5$ .

**Paso 3:  $[2, 2)$  — P2 despierta y *preempta* a P1.** En  $t = 2$ , P2 termina su WAIT:1 y vuelve a READY. Como P2 tiene mayor prioridad que P1, **preempta** a P1. P1 vuelve a READY con 1.5 unidades pendientes.

**Paso 4:  $[2, 3)$  — P2 ejecuta su segundo CPU:1.** P2 corre 1 unidad más y al llegar a  $t = 3$  ejecuta EXIT. P2 termina en  $t = 3$ .

**Paso 5:  $[3, 4.5)$  — P1 retoma y termina.** Con P2 fuera, la READY es [P1, idle]. P1 toma la CPU. Le faltan 1.5 unidades. El *quantum* es 2, así que no se agotará por tiempo. P1 corre 1.5 unidades y en  $t = 4.5$  hace EXIT.

## Diagrama de Gantt



## Métricas (turnaround)

| Tarea | Llegada | Fin               | Turnaround | CPU usada | Espera |
|-------|---------|-------------------|------------|-----------|--------|
| P2    | 0       | 3                 | 3          | 2         | 0      |
| P1    | 0       | 4.5               | 4.5        | 2.5       | 1      |
| idle  | 0       | — (nunca termina) | —          | —         | 4.5    |

### Respuesta

(b) **NO**. P1 *no* termina antes que P2:

- P2 termina en  $t = 3$ .
- P1 termina en  $t = 4.5$ .

**Razón:** aunque P2 tiene una ráfaga de CPU más corta (2 unidades en total vs. 2.5 de P1), su **prioridad superior** hace que siempre que esté **READY** preempte a P1. Cuando P2 va a **WAIT**, P1 progresa, pero cuando P2 despierta vuelve a quitarle la CPU.

## 5.c — ¿Cuándo se planifica la tarea idle?

La tarea idle (prio 0) solo es planificada cuando *no hay ninguna otra tarea* en **READY**:

- De  $t = 0$  a  $t = 4.5$  siempre hay alguna tarea de mayor prioridad (P1 o P2) en estado **READY** o **RUNNING**. Aunque P2 esté en **WAIT** durante  $[1, 2)$ , P1 ocupa la CPU en ese intervalo, así que idle no corre.
- En  $t = 3$  P2 termina. P1 sigue activa.
- En  $t = 4.5$  P1 termina. La cola **READY** queda solo con idle. Recién entonces idle toma la CPU.

### Respuesta

(c) idle se planifica desde  $t = 4.5$  en adelante, una vez que P1 y P2 han terminado y no hay otra tarea **READY**. Es la función clásica de idle: ocupar la CPU cuando no hay trabajo útil para hacer.

## Resumen del parcial

| # | Tema                       | Respuesta sintética                                                                                                                                   |
|---|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | V/F sobre SO               | a: V, b: F, c: F, d: V, e: F, f: V, g: F, h: F                                                                                                        |
| 2 | Linker estático            | a: F (dir. <code>f = 0x00FA</code> ); b: F (call externo); c: V (0x0184)                                                                              |
| 3 | fork vs threads            | A: dos órdenes posibles; B: tres interleavings, todos terminan con <code>main:2</code>                                                                |
| 4 | Deadlock                   | (a) Sí, ciclo $T_2 \rightarrow R_2 \rightarrow T_3 \rightarrow R_1 \rightarrow T_2$ ; T1 víctima colateral. (b) Sí, finalizar T2 desbloquea a T3 y T1 |
| 5 | Scheduling con prioridades | (a) [P2,P1,idle]; (b) NO: P2 termina en $t = 3$ , P1 en $t = 4.5$ ; (c) idle a partir de $t = 4.5$                                                    |